Obfuscating Game Hacking Through Hardware

Yannis Lagrosa, Bryan Deason, Braell Dotson, Lamar Dave

Sam Houston State University, Huntsville, Texas
Department of Computer Science

DFSC 4317 01 Information Security

Professor, Dr. Cihan Varol

Sam Houston State University
MEMBER THE TEXAS STATE UNIVERSITY SYSTEM

Outline

1. **Introduction**
2. Motivation
3. Related Work
4. Methodology
5. Result and Discussion
6. Conclusion
7. References

Introduction

Obfuscating Game Hacking Through Hardware – What does that mean..?

Our Target: Deadlock

- FPS/MOBA videogame
- Currently in a developmental phase (Invite-only)

Outline

1. Introduction
2. **Motivation**
3. Related Work
4. Methodology
5. Result and Discussion
6. Conclusion
7. References

Motivation

With the ever-growing popularity of e-sports, cheating has been difficult to monitor and avoid.

Our Goal

- Contribute to cheat detection methods.
- Make developers aware of potential vulnerabilities.
- Exploit vulnerabilities to create a radar-hack

By using industry-grade tools and methodologies.

Outline

1. Introduction
2. Motivation
3. **Related Work**
4. Methodology
5. Result and Discussion
6. Conclusion
7. References

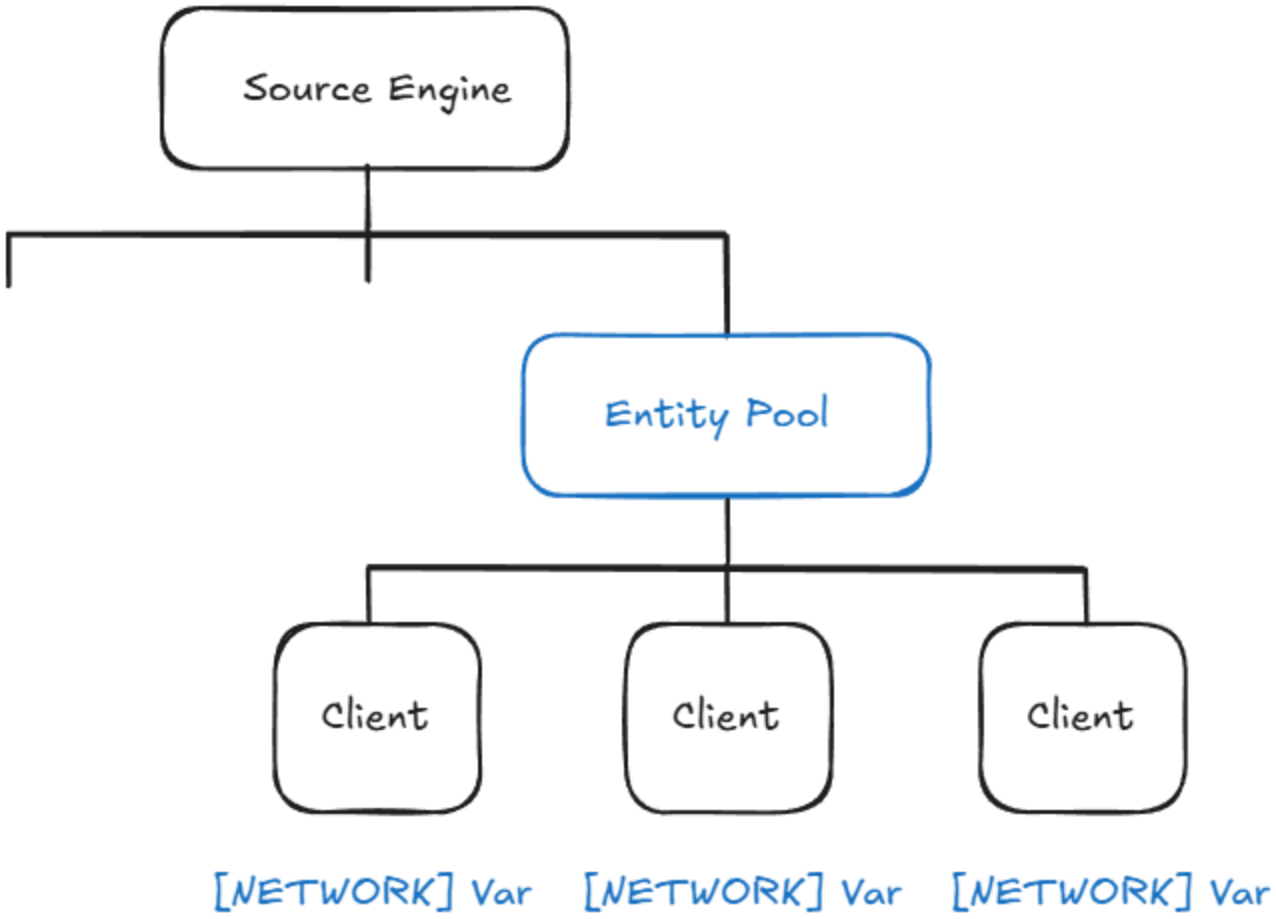
Related work

Authors(Year)	Focus of Study	Key Findings	Limitations
Quang Hoang William & Mary (2025)	Anti-cheat Software	Identified methods to improve anticheat for Assault Cube.	Did not discuss kernel memory bypassing.
Iantorno, Michael (2019)	Videogame Hacking	Identified videogame hacking practices and methodologies.	Did not discuss the utilization of hardware.
Caroline Andrea Rendenbach (2022)	Anti-Cheat Software	Dissected various anti-cheat technologies and their implementation in games.	Did not tackle usage of machine-learning in anti-cheats.
Patrick Stewin, Iurii Bystrov (2012)	DMA Attack	Explained and tested the feasibility of DMA-based Malware attacks.	Did not mention other cybercrimes possible through this technology.

Outline

1. Introduction
2. Motivation
3. Related Work
4. **Methodology**
5. Result and Discussion
6. Conclusion
7. References

Methodology



Methodology

Definition from Removing the Mystery from SEGMENT.

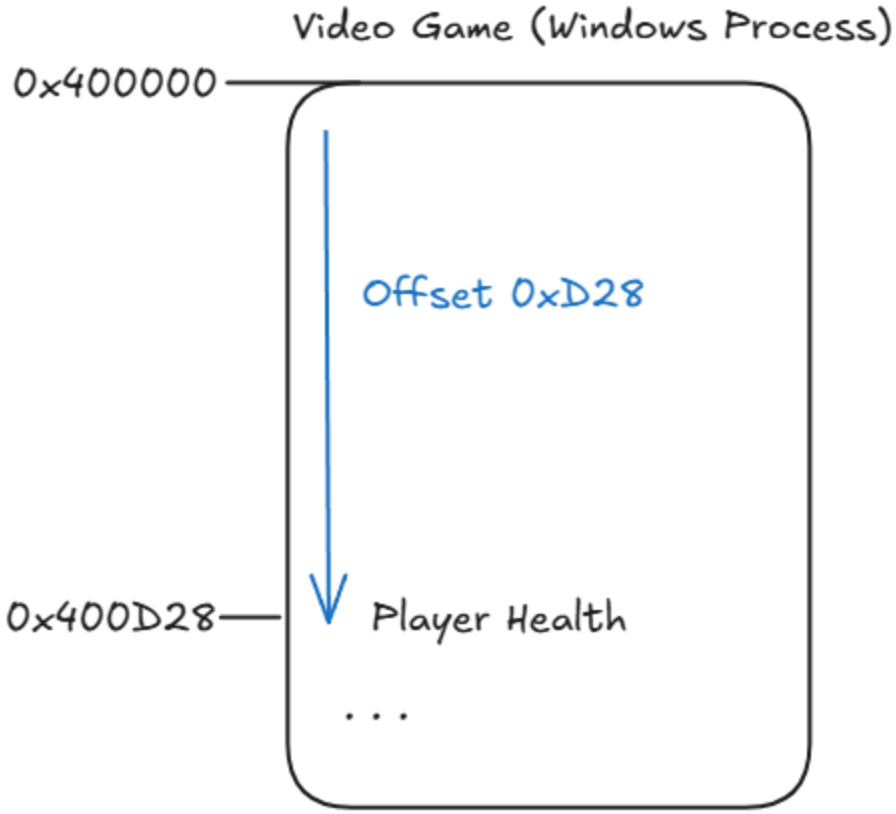
offset

/ˌɑːfˈset/

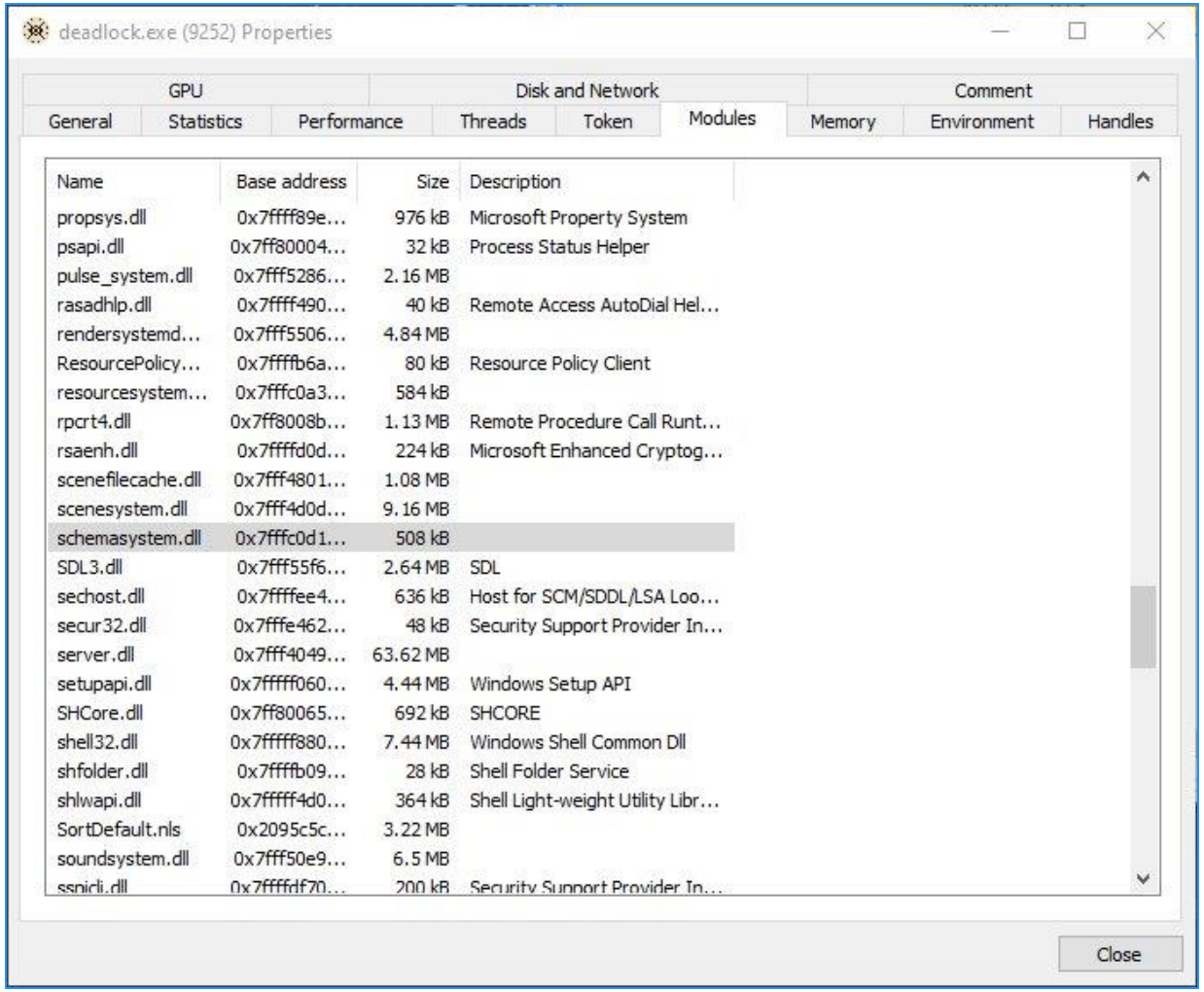
noun

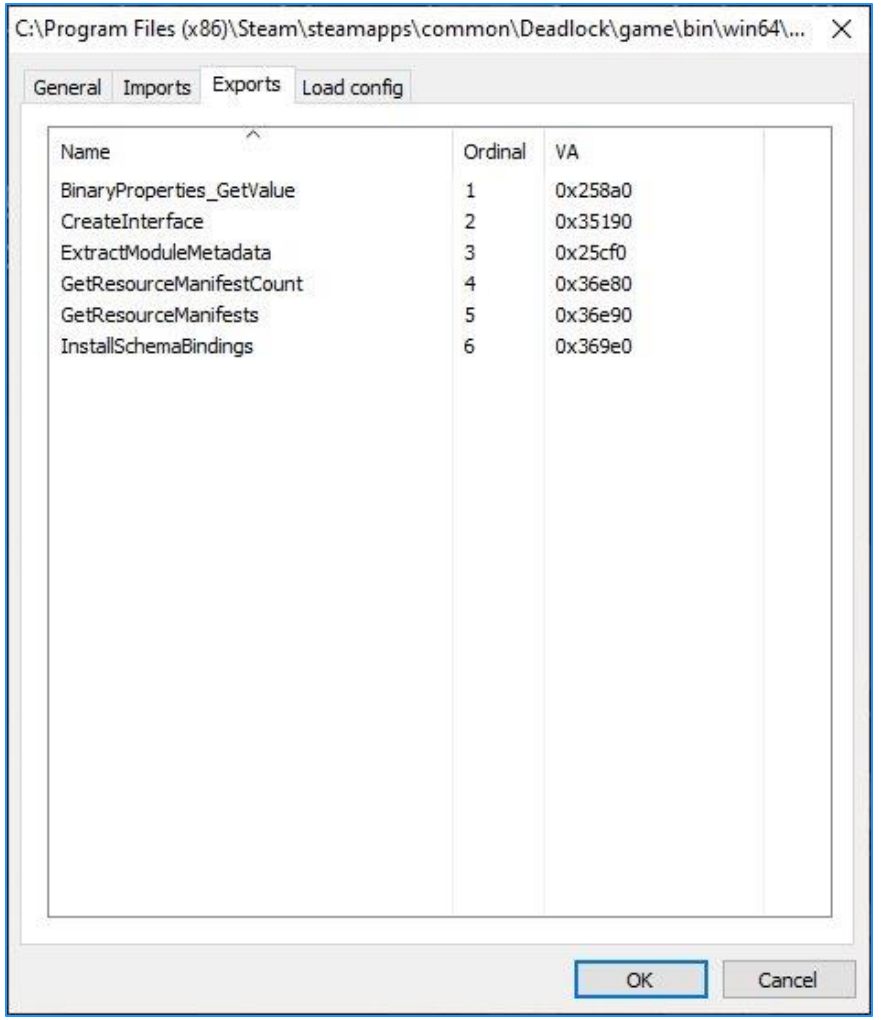
An offset is a distance from the start of a segment of memory to a certain variable.

- Allow us to bypass Address Space Layout Randomization (ASLR).
- Build paths to variables.



Methodology





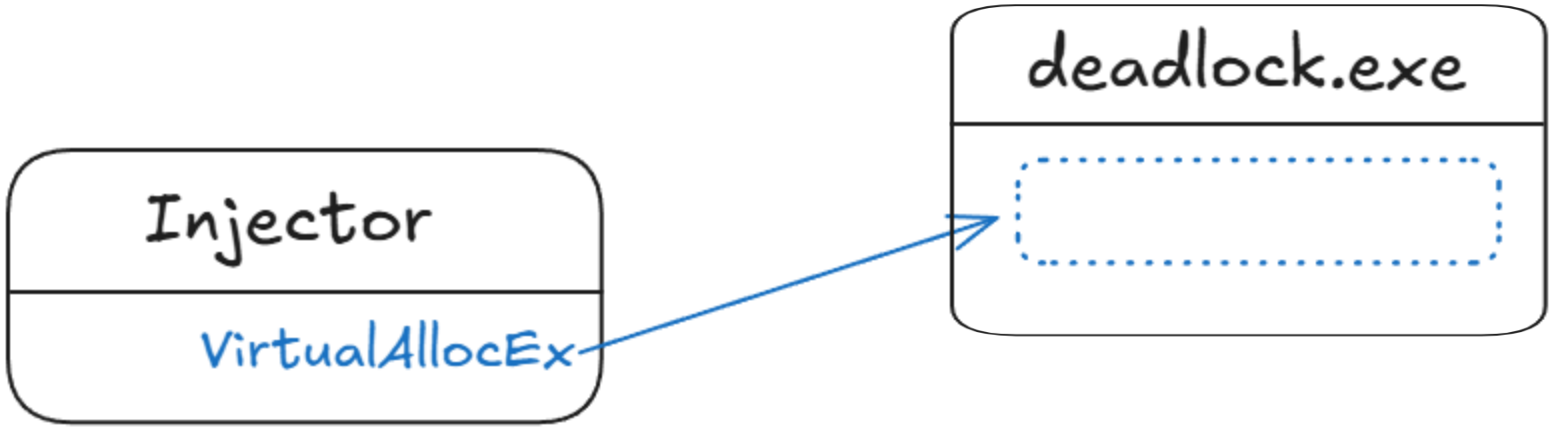
Methodology

Base	Module	Party	Path
00007FFFC89E0000	schemasystem.dll	User	C:\Pr

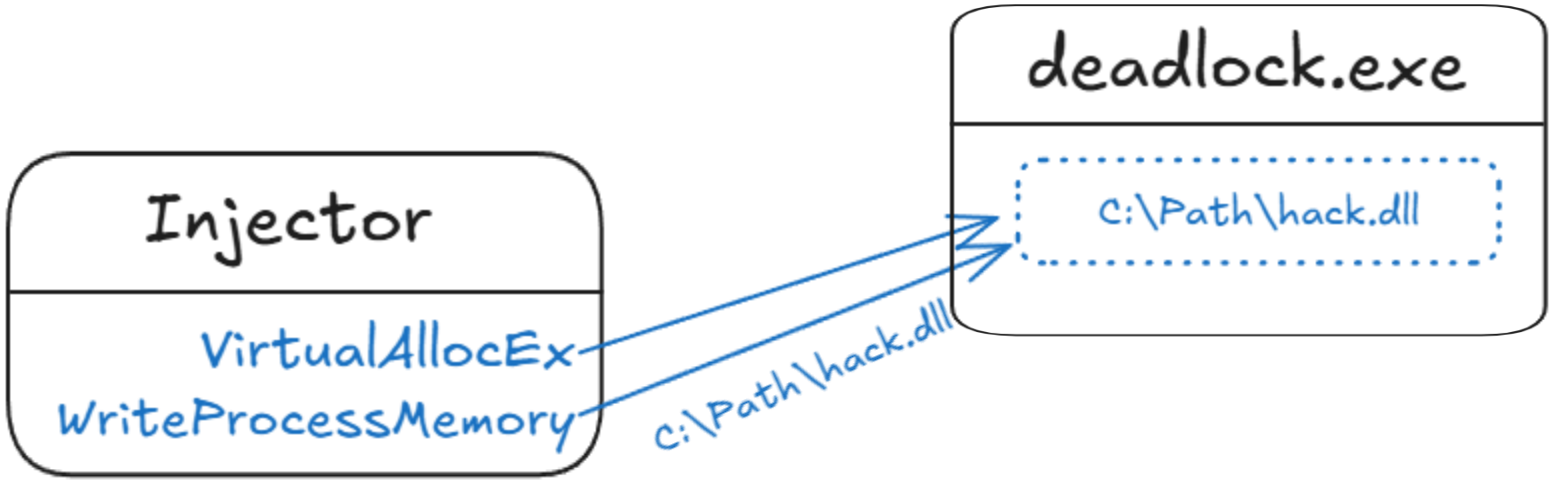
Address	Type	Ordinal	Symbol
00007FFFC8A064A0	Export	1	BinaryProperties_GetValue
00007FFFC8A068F0	Export	3	ExtractModuleMetadata
00007FFFC8A15D90	Export	2	CreateInterface
00007FFFC8A175E0	Export	6	InstallSchemaBindings
00007FFFC8A17A80	Export	4	GetResourceManifestCount
00007FFFC8A17A90	Export	5	GetResourceManifests
00007FFFC8A183CC	Export	0	OptionalHeader.AddressOfEntryPoint
00007FFFC8A28000	Import		kernel32_WriteConsoleW

```
00007FFFC92A5D6E    CC                  int3
00007FFFC92A5D6F    CC                  int3
00007FFFC92A5D70    4C:8941 08          mov qword ptr ds:[rcx+8],r8          rcx+08:"stem_001"
00007FFFC92A5D74    48:8911             mov qword ptr ds:[rcx],rdx           rcx:"SchemaSystem_001"
00007FFFC92A5D77    48:8B05 0AC30300    mov rax,qword ptr ds:[7FFFC92E2088]  rax:CreateInterface
00007FFFC92A5D7E    48:8941 10          mov qword ptr ds:[rcx+10],rax        rcx+10:GetResourceManifests+1D140, rax:CreateInterface
00007FFFC92A5D82    48:8BC1             mov rax,rcx                          rax:CreateInterface, rcx:"SchemaSystem_001"
00007FFFC92A5D85    48:890D FCC20300    mov qword ptr ds:[7FFFC92E2088],rcx  rcx:"SchemaSystem_001"
00007FFFC92A5D8C    C3                  ret
00007FFFC92A5D8D    CC                  int3
00007FFFC92A5D8E    CC                  int3
00007FFFC92A5D8F    CC                  int3
RIP RAX
00007FFFC92A5D90    4C:8B0D F1C20300    mov r9,qword ptr ds:[7FFFC92E2088]   CreateInterface
00007FFFC92A5D97    4C:8BD2             mov r10,rdx
00007FFFC92A5D9A    4C:8BD9             mov r11,rcx                          rcx:"SchemaSystem_001"
00007FFFC92A5D9D    4D:85C9             test r9,r9
00007FFFC92A5DA0    74 2E               je schemasystem.7FFFC92A5DD0
00007FFFC92A5DA2    49:8B41 08          mov rax,qword ptr ds:[r9+8]          rax:CreateInterface
00007FFFC92A5DA6    4D:8BC3             mov r8,r11
00007FFFC92A5DA9    4C:2BC0             sub r8,rax                           rax:CreateInterface
00007FFFC92A5DAC    0F1F40 00           nop dword ptr ds:[rax],eax
00007FFFC92A5DB0    0FB610              movzx edx,byte ptr ds:[rax]          rax:CreateInterface
00007FFFC92A5DB3    42:0FB60C00         movzx ecx,byte ptr ds:[rax+r8]       rax+r8*1:CreateInterface+1
00007FFFC92A5DB8    2BD1                sub edx,ecx
00007FFFC92A5DBA    75 07               jne schemasystem.7FFFC92A5DC3
00007FFFC92A5DBC    48:FFC0             inc rax                              rax:CreateInterface
```

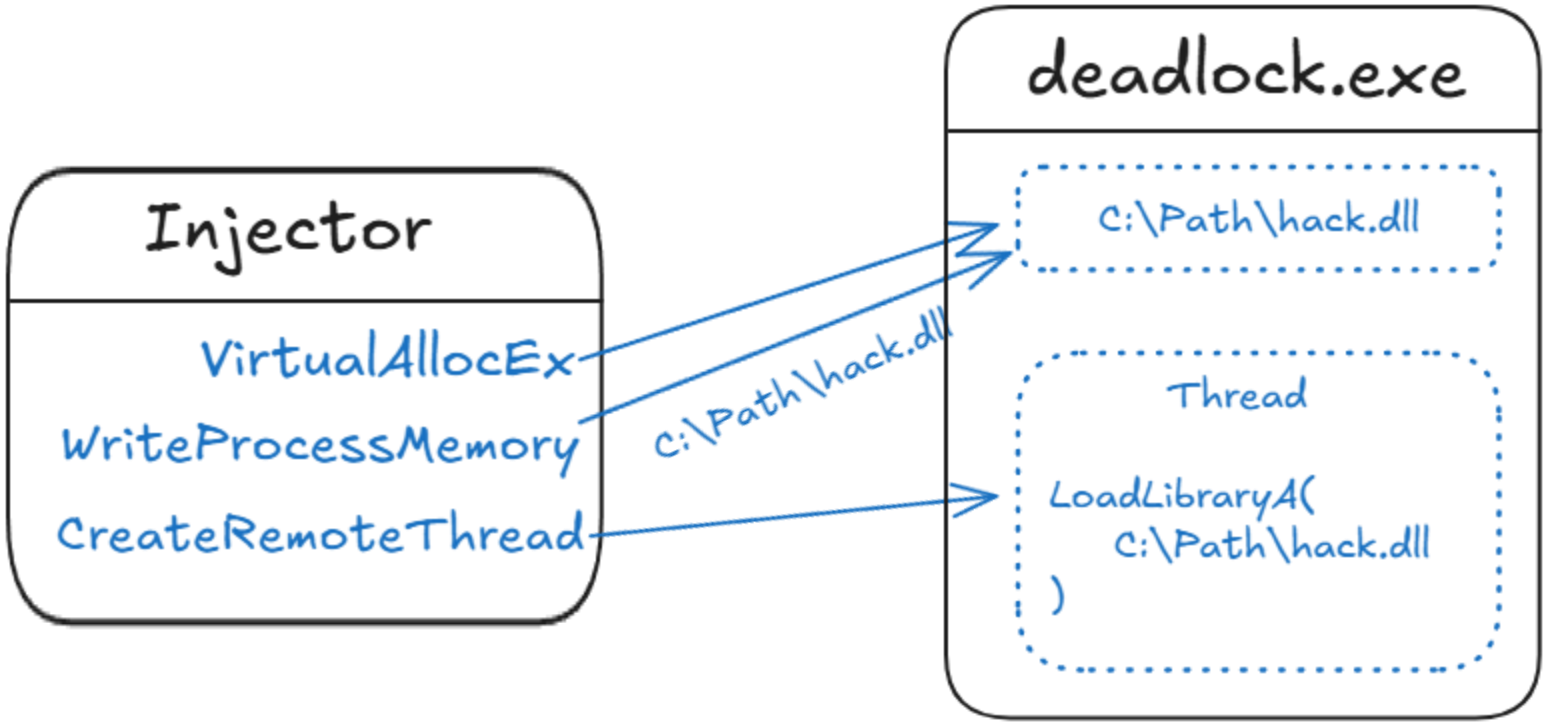
Methodology



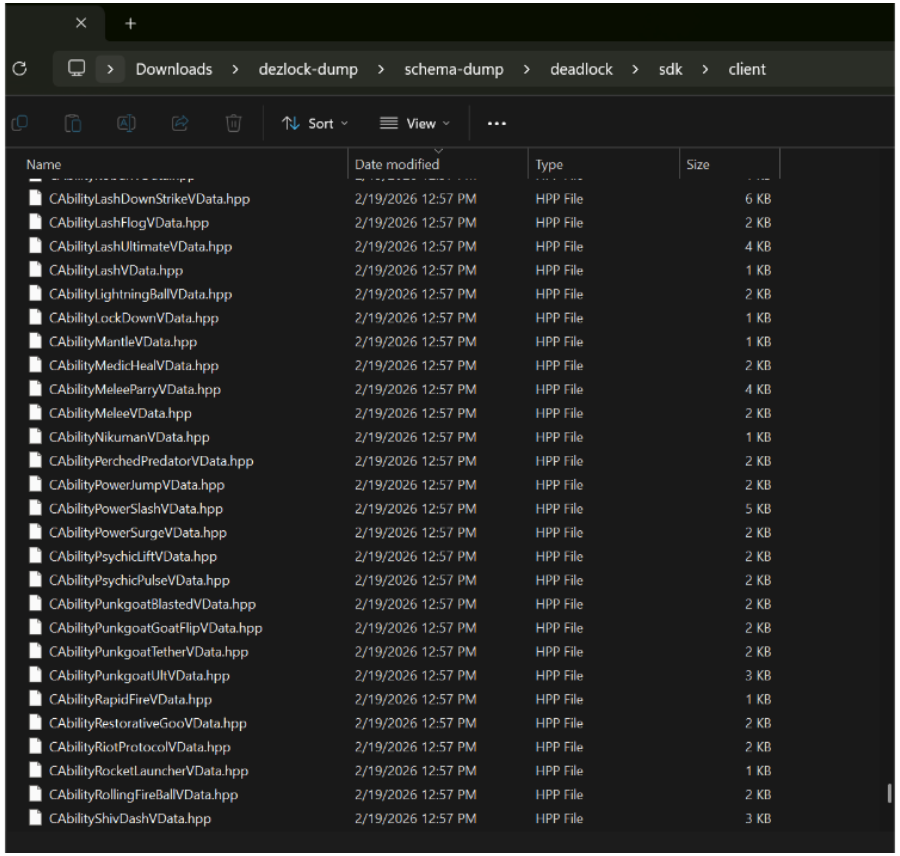
Methodology



Methodology



Methodology




```cpp
#pragma once
#ifndef DEZLOCK_SDK_CABILITY_FENCER_ULTIMATE_HPP
#define DEZLOCK_SDK_CABILITY_FENCER_ULTIMATE_HPP

#include <cstdint>
#include <cstddef>

struct CAbility_Fencer_Ultimate {
    uint8_t _pad0[0x1118];
    bool m_bCameraFinished; // 0x1118 (bool, 1)
    uint8_t _pad1119[0x3];
    float m_vCameraEndPos[3]; // 0x111C (Vector, 12)
    uint32_t m_hDummyForCamera; // 0x1128 (CHandle< C_BaseEntity >, 4)
    float m_bCameraLockedTime; // 0x112C (GameTime_t, 4)
    float m_flChaseCamStartTime; // 0x1130 (GameTime_t, 4)
    float m_vCameraChaseStartPos[3]; // 0x1134 (Vector, 12)
    float m_flActiveDashRangeWithHitTargets; // 0x1140 (float32, 4)
    float m_vStartPosition[3]; // 0x1144 (Vector, 12) [MNetworkEnable] [MNetworkUserGroup]
    float m_vDashDirection[3]; // 0x1150 (Vector, 12) [MNetworkEnable] [MNetworkUserGroup]
    float m_vecLastPosition[3]; // 0x115C (Vector, 12) [MNetworkEnable] [MNetworkUserGroup]
    bool m_bIsDashing; // 0x1168 (bool, 1) [MNetworkEnable] [MNetworkUserGroup]
    bool m_bIsFinishing; // 0x1169 (bool, 1) [MNetworkEnable] [MNetworkUserGroup]
    bool m_bStartedInAir; // 0x116A (bool, 1) [MNetworkEnable] [MNetworkUserGroup]
    bool m_bHitSomeone; // 0x116B (bool, 1) [MNetworkEnable] [MNetworkUserGroup]
    uint8_t _pad116C[0x4];
    uint8_t m_vecHitEnemies[0x18]; // 0x1170 (CUtlVector< CEntityIndex >)
    float m_flStuckTime; // 0x1188 (GameTime_t, 4)
    uint8_t m_UltHoldVFX[0x4]; // 0x118C (ParticleIndex_t)
    uint8_t m_DirPreviewVFX[0x4]; // 0x1190 (ParticleIndex_t)
    float m_qCurrDashCamAngle[3]; // 0x1194 (QAngle, 12)
    float m_flHitPctForCamera; // 0x11A0 (float32, 4)
    uint8_t _padEnd[0x624];
};

static_assert(sizeof(CAbility_Fencer_Ultimate) == 0x17C8);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_bCameraFinished) == 0x1118);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_vCameraEndPos) == 0x111C);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_hDummyForCamera) == 0x1128);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_bCameraLockedTime) == 0x112C);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_flChaseCamStartTime) == 0x1130);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_vCameraChaseStartPos) == 0x1134);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_flActiveDashRangeWithHitTargets) == 0x1140);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_vStartPosition) == 0x1144);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_vDashDirection) == 0x1150);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_vecLastPosition) == 0x115C);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_bIsDashing) == 0x1168);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_bIsFinishing) == 0x1169);
static_assert(offsetof(CAbility_Fencer_Ultimate, m_bStartedInAir) == 0x116A);
```

Methodology

Next Task: Find Entity List.

- Scan client.dll for an **Array of Bytes**.

What is an **Array of Bytes?**

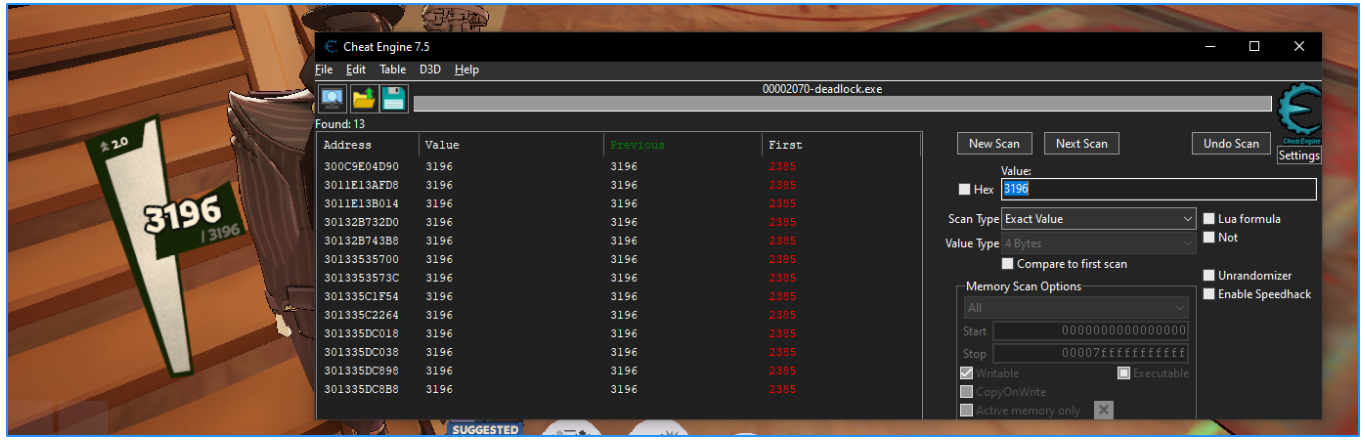
Address	Bytes	Opcode	
7FF72B870047	CD 21	**int**	21
7FF72B870049	B8 014CCD21	**mov**	eax,21CD4C01
7FF72B87004E	54	**push**	rsp
7FF72B87004F	68 69732070	**push**	70207369
7FF72B870054	72 6F	**jb**	7FF72B8700C5
7FF72B870056	67 72 61	**jb**	7FF72B8700BA

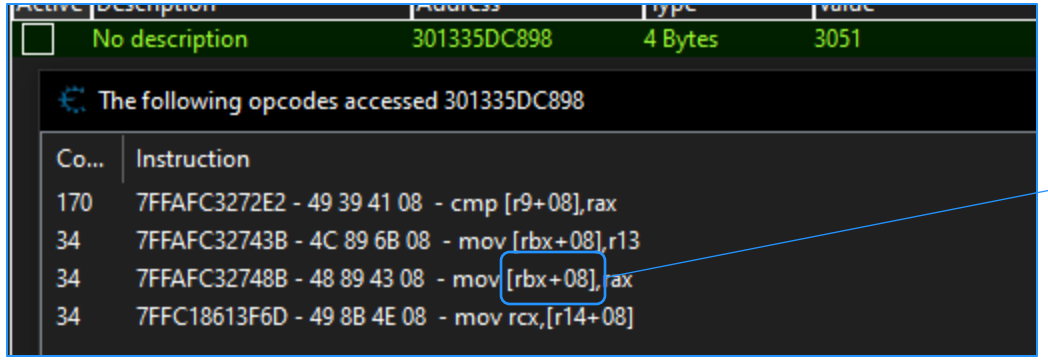
CD 21 B8 ?? ?? ?? ?? 54.
Array of Bytes (AOB) or "Pattern"

(Addresses are replaced with 'wild-cards')

Methodology

How do we find the AOB of our Entity List?





Remove offset to attain values stored in register.

Methodology

After scanning for our **Entity List**, we can dereference the values and build an algorithm to iterate through each of the entities.

```
0070 - Pointer to instance of CCitadelPlayerController    300A30700E8 : P->30133535C00
0078 - Pointer                                            300A30700F0 : P->7FFAF6A7E220
```

```
00E0 - Pointer to instance of CCitadelPlayerController    300A30700E8 : P->30133535C00
00E8 - Pointer                                            300A30700F0 : P->7FFAF6A7E220
```

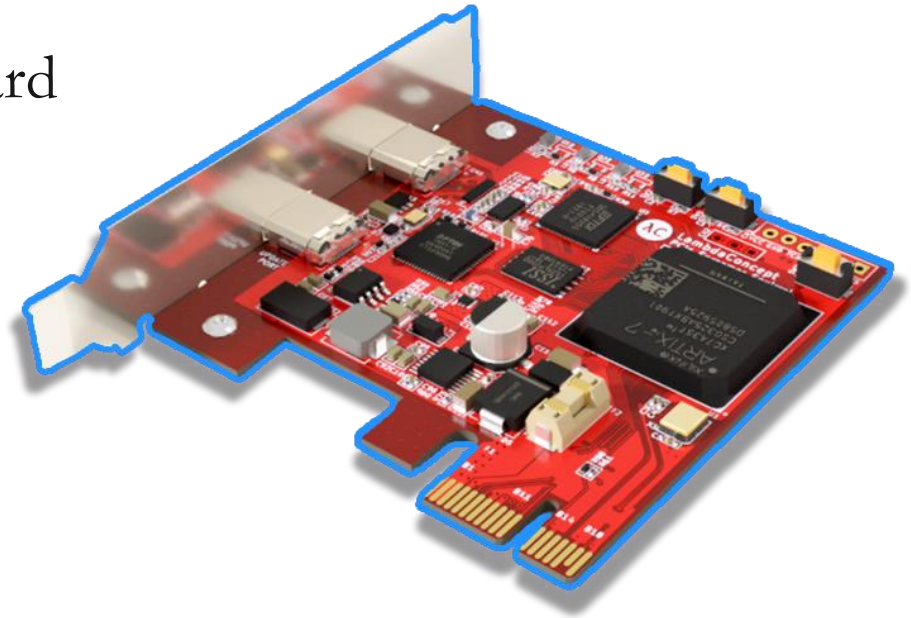
```
    0150 - Pointer to instance of CCitadelPlayerController    300A3070158 : P->30133536A00
    0158 - Pointer                                            300A3070160 : P->7FFAF6A7E220
```

- Padding of **0x70** between **CCitadelPlayerController** objects.

Methodology

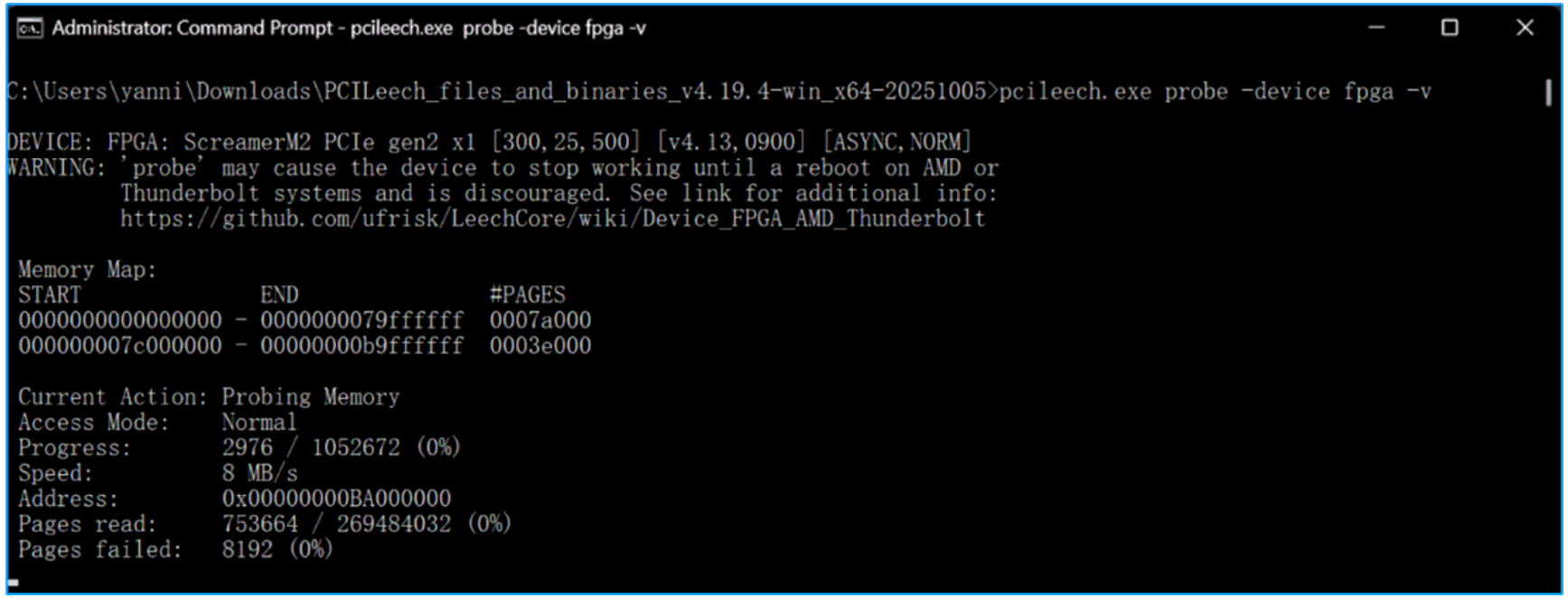
Screamer PCIe Squirrel, a circuit board designed to read/write memory via DMA.

- Accesses memory via PCIe Express Port on motherboard
- Device typically used for DMA attacks.
- Access memory without interfacing the kernel.



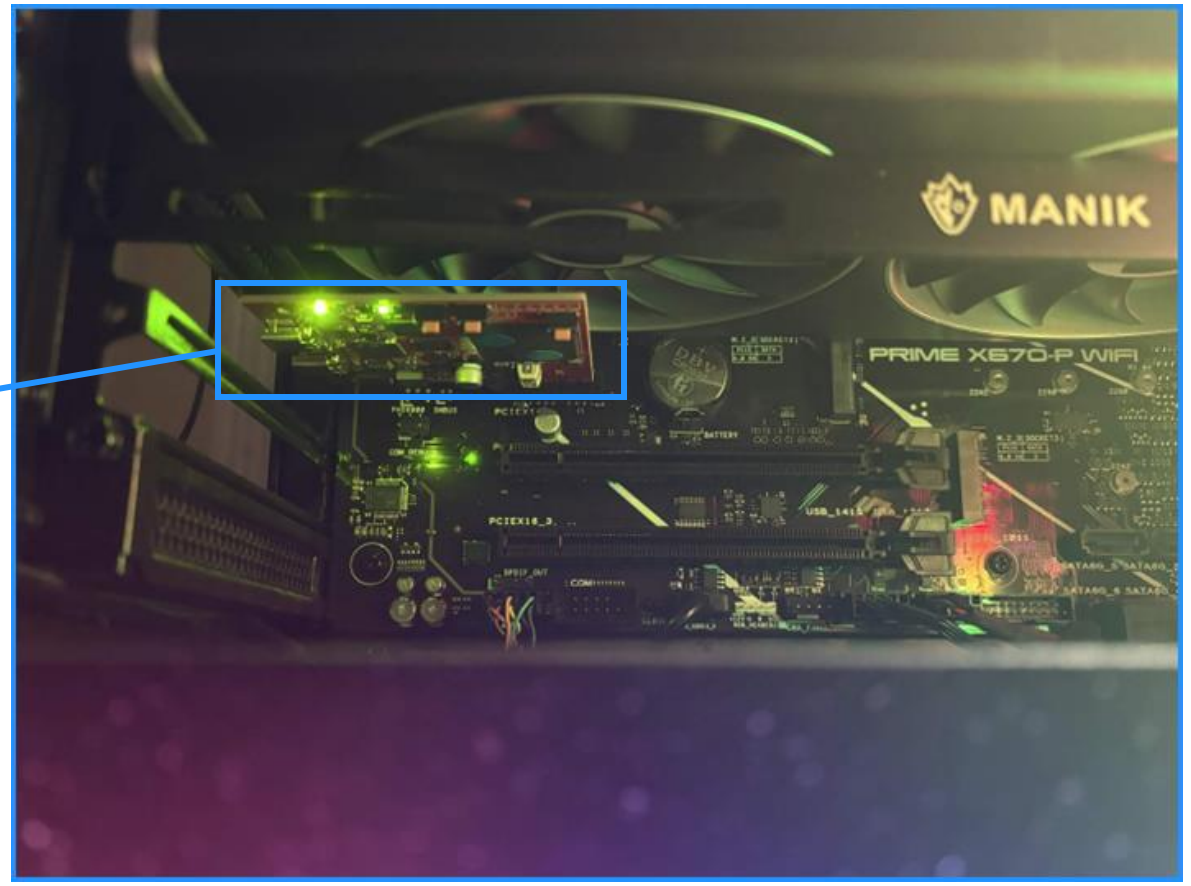
Methodology

Using **PCILeech** software to read memory from the target device.



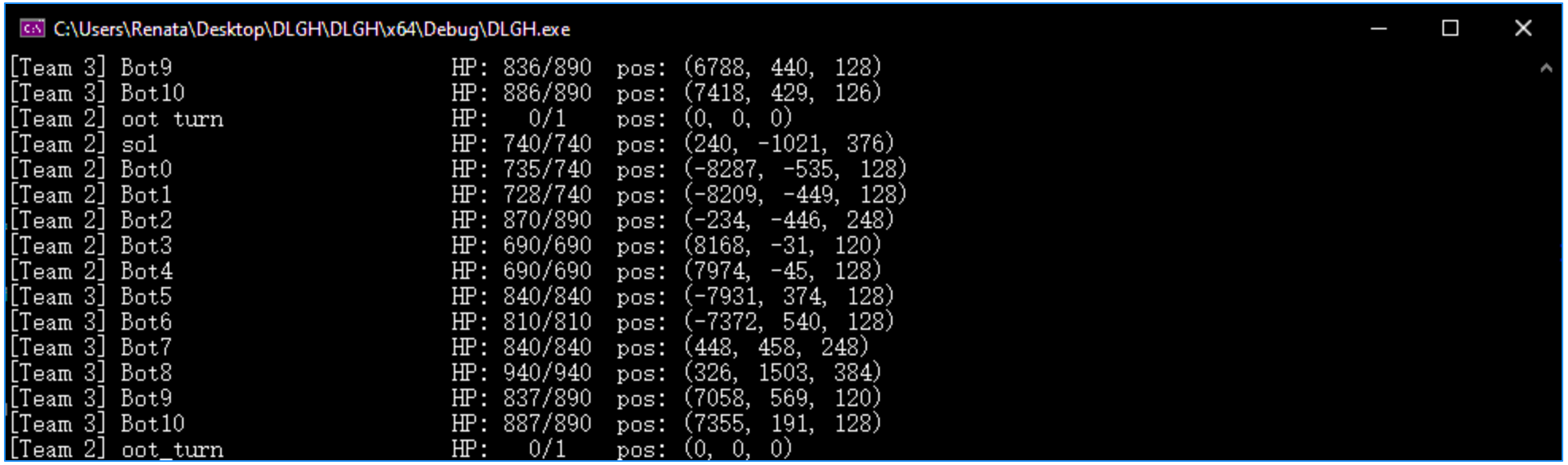
Methodology

DMA CARD



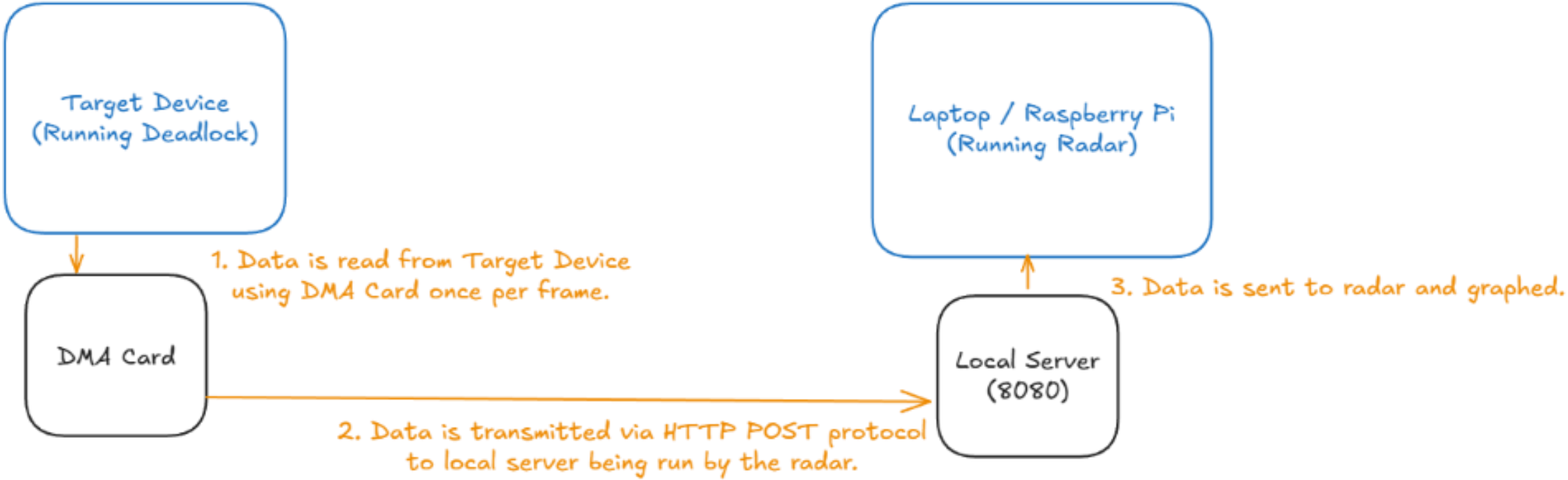
Methodology

Printing Entity Information to Console



Next Task: Build a Program to plot players on the map.

Minimap Program



```java
private void loop() {
    while (!glfwWindowShouldClose(window)) {

        glClear(GL_COLOR_BUFFER_BIT);

        glColor3f(0.1f,0.1f,0.1f); //minimap color
        glBegin(GL_QUADS); // minimap definition start
        glVertex2f(0.5f, -0.95f); //bottom left
        glVertex2f(0.5f, 0.0f); //top left
        glVertex2f(0.95f, 0.0f); //top right
        glVertex2f(0.95f, -0.95f); //bottom right
        glEnd();

        // --- DRAW ENEMIES ---
        for (float[] enemy : enemies) {
            if(enemy[0]== 0 && enemy[1] == 0) continue;
            drawEnemy(enemy[0], enemy[1]);
        }

        glfwSwapBuffers(window); // display what we drew
        glfwPollEvents();
    }
}
```

```java
private void drawEnemy(float x, float y) {
    float size = 0.01f;
    if (blueTeam) { y*= -1; x*=-1; } // Flip map
    // System.out.println("Drawing player at position: " + x + "," + y);

    glColor3f(1f, 0f, 0f); // red enemy

    glBegin(GL_QUADS); // size of "player/enemy" blocks
    glVertex2f(x - size, y - size);
    glVertex2f(x - size, y + size);
    glVertex2f(x + size, y + size);
    glVertex2f(x + size, y - size);
    glEnd();
}
```

OpenGL

Programming Stack

- OpenGL
- IntelliJ IDEA
- LWJGL framework

```
1   import org.lwjgl.opengl.GL;
2
3   import static org.lwjgl.glfw.GLFW.*;
4   import static org.lwjgl.opengl.GL11.*;
5   import static org.lwjgl.opengl.GL11C.GL_COLOR_BUFFER_BIT;
6   import static org.lwjgl.opengl.GL11C.glClear;
7   import static org.lwjgl.system.MemoryUtil.NULL;
8   import java.io.*;
9   import java.net.InetSocketAddress;
10  import com.sun.net.httpserver.HttpExchange;
11  import com.sun.net.httpserver.HttpHandler;
12  import com.sun.net.httpserver.HttpServer;
13
```

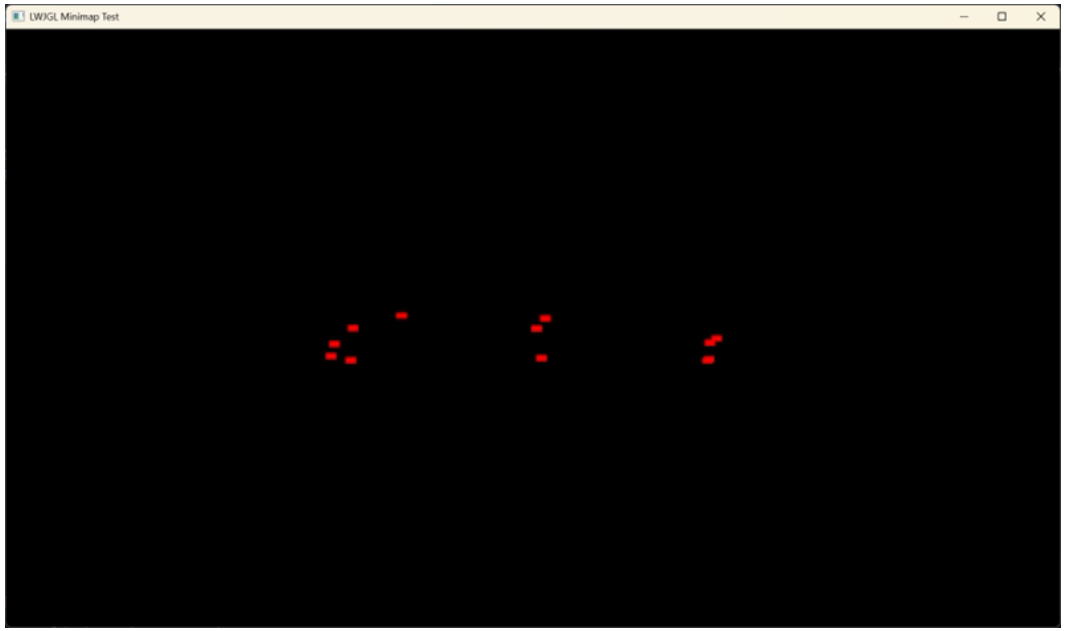
Webserver

Data gathered from DMA Card is read by web-server.

Data is parsed and graphed.

```
static class MyHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange t) throws IOException {
        InputStream in = t.getRequestBody();
        String readLine = "";
        BufferedReader br = new BufferedReader(new InputStreamReader(in));
        // System.out.println(br.readLine());
        // Definitely should abstract this into a method
        float[][] newenemies = new float[14][2];
        int count = 0;
        while (((readLine = br.readLine()) != null)) {
            System.out.println(readLine);
            readLine = readLine.substring(2);
            blueTeam = readLine.charAt(0) == '3';
            System.out.println("Team is. . ." + blueTeam);
            readLine = readLine.substring(1, readLine.length()-1);
            String[] splitLines = readLine.split("\\+");
            for(String strings : splitLines){
                String[] newstring = strings.split(",");
                newenemies[count][0] = Float.parseFloat(newstring[0])/20000;
                newenemies[count][1] = Float.parseFloat(newstring[1])/20000;
                count++;
            }
            enemies = newenemies;
        }
        String response = "";
        t.sendResponseHeaders(200, response.length());
        OutputStream os = t.getResponseBody();
        os.write(response.getBytes());
        os.close();
    }
}
```

Methodology



Radar running on PC 2.



Radar overlayed onto PC 1.



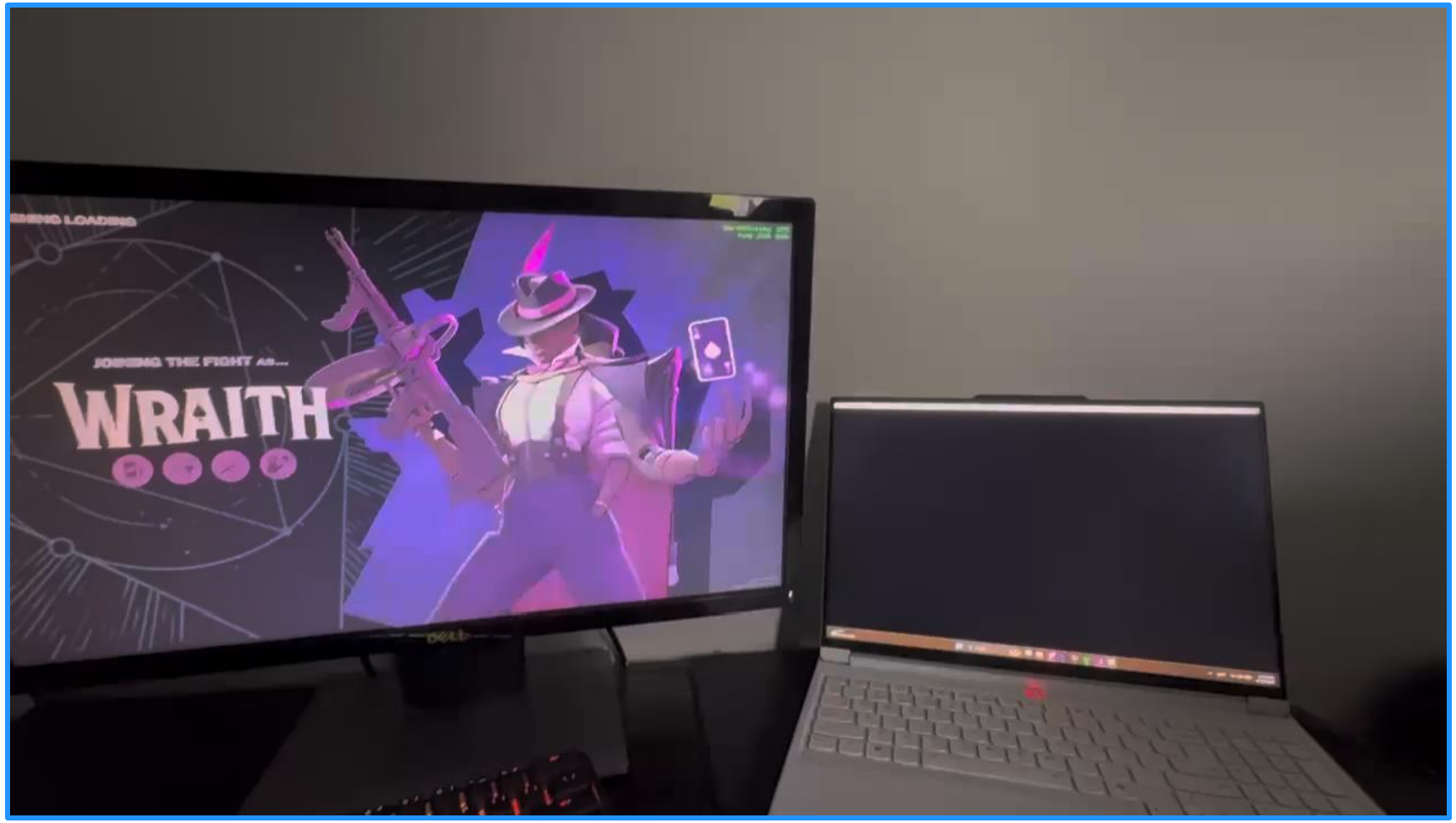
PC1 In-Game Map.

Outline

1. Introduction
2. Motivation
3. Related Work
4. Methodology
5. **Result and Discussion**
6. Conclusion
7. References

Result and Discussion

Demonstration



Result and Discussion

Implementation of the project was a **success**, we were successfully able to:

- **Reverse engineer** the game
- **Identify offsets** for various player entities and classes,
- and most importantly, **externalize** our cheat via hardware.

Result and Discussion

Through our analysis of the Deadlock program and its DLL files, we were able to identify the following vulnerabilities in the game.

Vulnerability	How we manipulated it	Suggestions to fix it
SchemaSystem reveals all offsets to client.	This vulnerability allowed our team to analyze the memory of the game and reveal information stored only in the code.	Improve injected DLL detection via an anti-cheat software or module. Monitor process memory.
Lack of Reverse Engineering counter-measures	Our team was able to identify critical memory addresses used to develop the cheat.	Monitor WinAPI function calls. Terminate the process if Windows attempts to debug it.
Lack of Peripheral Monitoring	Our team was able to use a DMA card to read memory from the game.	Monitor peripherals, especially the PCIe port. Scan for known DMA firmware patterns/strings.

Outline

1. Objective
2. Introduction
3. Related Work
4. Methodology
5. Result and Discussion
6. **Conclusion**
7. References

Conclusion

Although we did offer suggestions to fix some of these vulnerabilities, ultimately cheat developers and game developers will always be at war with one-another. It will be impossible to guarantee a completely neutral game environment.

It is still within the best interest of Valve to improve upon their anti-cheat though. If cheating requires set-up such as hardware, it intrinsically becomes more exclusive, meaning there will be less cheaters on the servers.

We hope to share our findings with the creators of this game to improve its development cycle.

Future work

- Height indicator
- Adding player icons instead of red boxes
- Overlaying our map onto the minimap in game

Outline

1. Introduction
2. Motivation
3. Related Work
4. Methodology
5. Result and Discussion
6. Conclusion
7. **References**

References

Aumaitre, D., Devine, C.: Subverting Windows 7 x64 Kernel with DMA attacks. Sogeti ESEC Lab (July 2010), http://esec-lab.sogeti.com/dotclear/public/publications/10-hitbamsterdam-dmaattacks.pdf

Boileau, A.: Hit by a Bus: Physical Access Attacks with Firewire. Security-Assessment.com, Ruxcon 2006 (October 2006), http://www.security-assessment.com/files/presentations/ab_firewire_rux2k6-final.pdf

Ubisoft. Fairfight anti-cheat software. Sept. 2021. https://www.i3d.net/products/hosting/anti-cheat-software/.

Steam. Valve Anti-Cheat-System (VAC). https:// help.steampowered.com/de/faqs/view /571A-97DA-70E9-FF74#consequences.

Steampowered. VAC-enabled Steam games. https://store.steampowered.com/search/?category1=998&category2=8.

Steamworks. Vac Integration (steamworks documentation). https://partner.steamgames.com/doc/features/anticheat/vac _ integration.

Thank You for your Attention!